.Union

.ndarray

T: ndarray
base: NoneType
ctypes: NoneType
data: NoneType
dtype: NoneType
flags: NoneType
flat: ndarray
imag: ndarray
itemsize: NoneType
nbytes: NoneType
ndim: NoneType
real: ndarray
shape: ndarray
size: NoneType
strides: NoneType

__abs__()
__add__(value)
__and__(value)
__array__(dtype)
__array_wrap__(obj)
__class_getitem__(value)
__contains__(key)
__copy__()
__deepcopy__(memo)
__divmod__(value)
__eq__(value)
__float__()
__floordiv__()
__ge__(value)
__getitem__(key)
__gt__(value)
__iadd__(value)
__iand__(value)
__ifloordiv__(value)
__ilshift__(value)
__imod__(value)
__imul__(value)
__init__(shape, dtype, buffer, offset, strides, order)
__int__()
__invert__()
__ior__(value)
__ipow__(value)
__irshift__(value)
__isub__(value)
__itruediv__(value)
__ixor__(value)
__le__(value)
__len__()
__lshift__(value)
__lt__(value)
__matmul__(value)
__mod__(value)
__mul__(value)
__ne__(value)
__neg__()
__or__(value)
__pos__()
__pow__()
__repr__()
__rshift__()
__setitem__(key, value)
__str__()
__sub__(value)
__truediv__(value)
__xor__(value)
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)

pandas.core.indexes.numeric.NumericIndex

_can_hold_strings: bool
_default_dtype
_dtype_validation_metadata: tuple[Callable[..., bool], str]
_engine_type
_engine_types: dict[np.dtype, type[libindex.IndexEngine]]
_index_descr_args: dict
_is_all_dates
_is_numeric_dtype: bool
_values

__contains__(key): bool
__new__(data, dtype: Dtype | None, copy, name)
_assert_safe_casting(data: np.ndarray, subarr: np.ndarray): None
_can_hold_na(): bool
_convert_slice_indexer(key: slice, kind: str)
_convert_tolerance(tolerance, target)
_ensure_array(data, dtype, copy: bool)
_ensure_dtype(dtype: Dtype | None): np.dtype | None
_format_native_types(na_rep, float_format, decimal, quoting)
_is_comparable_dtype(dtype: DtypeObj): bool
_maybe_cast_slice_bound(label, side: str, kind)
_shallow_copy(values, name: Hashable)
_should_fallback_to_positional(): bool
_validate_dtype(dtype: Dtype | None): None
astype(dtype, copy)
inferred_type(): str

_values

DynRephrESAn.DynRephAnMainInterface

__cmp_corpora_menu

__rephrase_xlsx: str
__single_corpora_menu

__ComparativeCorporaMenuLoader(dataDic: dict[str:pd.DataFrame()], anType: str): CmpCorpusMenu
__MainPage()
__SingleCorporaMenuLoader(dataDic: dict[str:pd.DataFrame()], submenu_prefix: str, anType: str): SingleCorpusMenu
__init__(): None
__load_data(dir_address: str): dict[str:pd.DataFrame()]
__style_css(file)

data_display.display_corpora_cmp.ComparativeCorporaSimple

__cf: Dict[str,Any], dict
__dataDic: dict[str, pd.DataFrame()]
__note0: type
__note1: type
prefixCtr: int

Distribution()
Ngrams()
PoS()
WordCloud()
__Display(data_dic: dict[str, pd.DataFrame()], classType: Any): Any
__init__(data_dic: dict[str, pd.DataFrame()], config: Dict[str,Any])
__loadFilteredDataToDic(): Tuple[Dict[str,Any], Dict[str,Any]]
__updateCfg(config: Dict[str,Any]): None
noteClass()

submenus.comparative_corpus.CmpCorpusMenu

__ADUorSpeaker: str
__anCf: dict
__anType: str
__dataDic: dict
__dataDict
__dataLoaders: list[SingleCorpusMenu]
__display
__keyDic: dict
__tabLabels: list[str]

__init__(dataDict: pd, anType: str)
__updateCfg(config: Dict[str,Any])
clearTabsSelections(): None
display(units)

__display

numpy.core.numerictypes.dtype

alignment: NoneType
base: NoneType
byteorder: NoneType
char: NoneType
descr: NoneType
fields: NoneType
flags: NoneType
hasobject: NoneType
isalignedstruct: NoneType
isbuiltin: NoneType
isnative: NoneType
itemsize: NoneType
kind: NoneType
metadata: NoneType
name: NoneType
names: NoneType, list, tuple
num: NoneType
shape: NoneType
str: NoneType
subdtype: NoneType
type: NoneType

__init__(obj, align, copy)
__neg__()
newbyteorder(new_order)

_default_dtype

pandas.core.accessor.CachedAccessor

_accessor
_name: str

__get__(obj, cls)
__init__(name: str, accessor): None

pandas.core.frame.DataFrame

T

_AXIS_LEN: int
_AXIS_NAMES
_AXIS_NUMBERS
_AXIS_ORDERS: list
_AXIS_REVERSED: bool
_AXIS_TO_AXIS_NUMBER: dict[Axis, int]
_HANDLED_TYPES: tuple
_accessors: set[str]
_agg_examples_doc
_agg_summary_and_see_also_doc
_can_fast_transpose
_constructor
_constructor_sliced: type[Series]
_hidden_attrs: frozenset[str]
_info_axis_name: str
_info_axis_number: int
_internal_names_set
_is_homogeneous_type
_logical_method
_mgr: BlockManager | ArrayManager
_series
_typ: str
_values
agg
axes

boxplot
columns
hist
index
plot
shape
sparse

style
values

__divmod__(other): tuple[DataFrame, DataFrame]
__getitem__(key)
__init__(data, index: Axes | None, columns: Axes | None, dtype: Dtype | None, copy: bool | None)
__len__(): int
__matmul__(other: Series): Series
__rdivmod__(other): tuple[DataFrame, DataFrame]
__repr__(): str
__rmatmul__(other)
__setitem__(key, value)
_arith_method(other, op)
_box_col_values(values, loc: int): Series
_clear_item_cache(): None
_cmp_method(other, op)
_combine_frame(other: DataFrame, func, fill_value)
_construct_result(result): DataFrame
_count_level(level: Level, axis: int, numeric_only: bool)
_dispatch_frame_op(right, func: Callable, axis: int | None)
_ensure_valid_index(value): None
_from_arrays(arrays, columns, index, dtype: Dtype | None, verify_integrity: bool): DataFrame
_get_agg_axis(axis_num: int): Index
_get_column_array(i: int): ArrayLike
_get_item_cache(item: Hashable): Series
_get_value(index, col, takeable: bool): Scalar
_getitem_bool_array(key)
_getitem_multilevel(key)
_gotitem(key: IndexLabel, ndim: int, subset: FrameOrSeriesUnion | None): FrameOrSeriesUnion
_info_repr(): bool
_iset_item(loc: int, value): None
_iset_item_mgr(loc: int | slice | np.ndarray, value): None
_iset_not_inplace(key, value)
_iter_column_arrays(): Iterator[ArrayLike]
_ixs(i: int, axis: int)
_join_compat(other: FrameOrSeriesUnion, on: IndexLabel | None, how: str, lsuffix: str, rsuffix: str, sort: bool)
_maybe_cache_changed(item, value: Series): None
_reduce(op, name: str)
_reindex_axes(axes, level, limit, tolerance, method, fill_value, copy)
_reindex_columns(new_columns, method, copy: bool, level: Level, fill_value, limit, tolerance)
_reindex_index(new_index, method, copy: bool, level: Level, fill_value, limit, tolerance)
_reindex_multi(axes, copy: bool, fill_value): DataFrame
_replace_columnwise(mapping: dict[Hashable, tuple[Any, Any]], inplace: bool, regex)
_repr_fits_horizontal_(ignore_width: bool): bool
_repr_fits_vertical_(): bool
_repr_html_(): str | None
_reset_cacher(): None
_sanitize_column(value): ArrayLike
_set_item(key, value): None
_set_item_frame_value(key, value: DataFrame): None
_set_item_mgr(key, value: ArrayLike): None
_set_value(index: IndexLabel, col, value: Scalar, takeable: bool): None
_setitem_array(key, value)
_setitem_frame(key, value)
_setitem_slice(key: slice, value)
_to_dict_of_blocks(copy: bool)
aggregate(func, axis: Axis)
align(other, join: str, axis: Axis | None, level: Level | None, copy: bool, fill_value, method: str | None, limit, fill_axis: Axis, broadcast_axis: Axis | None): DataFrame
append(other, ignore_index: bool, verify_integrity: bool, sort: bool): DataFrame
apply(func: AggFuncType, axis: Axis, raw: bool, result_type, args)
applymap(func: PythonFuncType, na_action: str | None): DataFrame
asfreq(freq: Frequency, method, how: str | None, normalize: bool, fill_value): DataFrame
assign(): DataFrame
bfill(axis: None | Axis, inplace: bool, limit: None | int, downcast): DataFrame | None
clip(lower, upper, axis: Axis | None, inplace: bool): DataFrame | None
combine(other: DataFrame, func, fill_value, overwrite: bool): DataFrame
combine_first(other: DataFrame): DataFrame
compare(other: DataFrame, align_axis: Axis, keep_shape: bool, keep_equal: bool): DataFrame
corr(method: str | Callable[[np.ndarray, np.ndarray], float], min_periods: int): DataFrame
corrwith(other, axis: Axis, drop, method): Series
count(axis: Axis, level: Level | None, numeric_only: bool)
cov(min_periods: int | None, ddof: int | None): DataFrame
diff(periods: int, axis: Axis): DataFrame
dot(other: Series): Series
drop(labels, axis: Axis, index, columns, level: Level | None, inplace: bool, errors: str)
drop_duplicates(subset: Hashable | Sequence[Hashable] | None, keep: Literal['first'] | Literal['last'] | Literal[False], inplace: bool, ignore_index: bool): DataFrame | None
dropna(axis: Axis, how: str, thresh, subset, inplace: bool)
duplicated(subset: Hashable | Sequence[Hashable] | None, keep: Literal['first'] | Literal['last'] | Literal[False]): Series
eval(expr: str, inplace: bool)
explode(column: IndexLabel, ignore_index: bool): DataFrame
ffill(axis: None | Axis, inplace: bool, limit: None | int, downcast): DataFrame | None
fillna(value, method: FillnaOptions | None, axis: Axis | None, inplace: Literal[False], limit, downcast): DataFrame
from_dict(data, orient: str, dtype: Dtype | None, columns): DataFrame
from_records(data, index, exclude, columns, coerce_float: bool, nrows: int | None): DataFrame
groupby(by, axis: Axis, level: Level | None, as_index: bool, sort: bool, group_keys: bool, squeeze: bool | lib.NoDefault, observed: bool, dropna: bool): DataFrameGroupBy
idxmax(axis: Axis, skipna: bool): Series
idxmin(axis: Axis, skipna: bool): Series
info(verbose: bool | None, buf: IO[str] | None, max_cols: int | None, memory_usage: bool | str | None, show_counts: bool | None, null_counts: bool | None): None
insert(loc, column, value, allow_duplicates: bool): None
interpolate(method: str, axis: Axis, limit: int | None, inplace: bool, limit_direction: str | None, limit_area: str | None, downcast: str | None): DataFrame | None
isin(values): DataFrame
isna(): DataFrame
isnull(): DataFrame
items(): Iterable[tuple[Hashable, Series]]
iteritems(): Iterable[tuple[Hashable, Series]]
iterrows(): Iterable[tuple[Hashable, Series]]
itertuples(index: bool, name: str | None): Iterable[tuple[Any,...]]
join(other: FrameOrSeriesUnion, on: IndexLabel | None, how: str, lsuffix: str, rsuffix: str, sort: bool): DataFrame
lookup(row_labels: Sequence[IndexLabel], col_labels: Sequence[IndexLabel]): np.ndarray
mask(cond, other, inplace, axis, level, errors, try_cast)
melt(id_vars, value_vars, var_name, value_name, col_level: Level | None, ignore_index: bool): DataFrame
memory_usage(index: bool, deep: bool): Series
merge(right: FrameOrSeriesUnion, how: str, on: IndexLabel | None, left_on: IndexLabel | None, right_on: IndexLabel | None, left_index: bool, right_index: bool, sort: bool, suffixes: Suffixes, copy: bool, indicator: bool, validate: str | None): DataFrame mode(axis: Axis, numeric_only: bool, dropna: bool): DataFrame
nlargest(n, columns, keep: str): DataFrame
notna(): DataFrame
notnull(): DataFrame
nsmallest(n, columns, keep: str): DataFrame
nunique(axis: Axis, dropna: bool): Series
pivot(index, columns, values): DataFrame
pivot_table(values, index, columns, aggfunc, fill_value, margins, dropna, margins_name, observed, sort): DataFrame
pop(item: Hashable): Series
quantile(q, axis: Axis, numeric_only: bool, interpolation: str)
query(expr: str, inplace: bool)
reindex(): DataFrame
rename(mapper: Renamer | None): DataFrame | None
reorder_levels(order: Sequence[Axis], axis: Axis): DataFrame
replace(to_replace, value, inplace: bool, limit, regex: bool, method: str)
resample(rule, axis, closed: str | None, label: str | None, convention: str, kind: str | None, loffset, base: int | None, on, level, origin: str | TimestampConvertibleTypes, offset: TimedeltaConvertibleTypes | None): Resampler reset_index(level: Hashable | Sequence[Hashable] | None, drop: bool, inplace: Literal[False], col_level: Hashable, col_fill: Hashable): DataFrame
round(decimals: int | dict[IndexLabel, int] | Series): DataFrame
select_dtypes(include, exclude): DataFrame
set_axis(labels, axis: Axis, inplace: Literal[False]): DataFrame
set_index(keys, drop: bool, append: bool, inplace: bool, verify_integrity: bool)
shift(periods, freq: Frequency | None, axis: Axis, fill_value): DataFrame
sort_index(axis: Axis, level: Level | None, ascending: bool | int | Sequence[bool | int], inplace: bool, kind: str, na_position: str, sort_remaining: bool, ignore_index: bool, key: IndexKeyFunc)
sort_values(by, axis: Axis, ascending, inplace: bool, kind: str, na_position: str, ignore_index: bool, key: ValueKeyFunc)
stack(level: Level, dropna: bool)
swaplevel(i: Axis, j: Axis, axis: Axis): DataFrame
to_dict(orient: str, into)
to_feather(path: FilePathOrBuffer[AnyStr]): None
to_gbq(destination_table: str, project_id: str | None, chunksize: int | None, reauth: bool, if_exists: str, auth_local_webserver: bool, table_schema: list[dict[str, str]] | None, location: str | None, progress_bar: bool, credentials): None
to_html(buf: FilePathOrBuffer[str] | None, columns: Sequence[str] | None, col_space: ColspaceArgType | None, header: bool | Sequence[str], index: bool, na_rep: str, formatters: FormattersType | None, float_format: FloatFormatType | None, sparsify: bool | None, index_names: bool, justify: str | None, max_rows: int | None, max_cols: int | None, show_dimensions: bool | str, decimal: str, bold_rows: bool, classes: str | list | tuple | None, escape: bool, notebook: bool, border: int | None, table_id: str | None, render_links: bool, encoding: str | None) to_markdown(buf: IO[str] | str | None, mode: str, index: bool, storage_options: StorageOptions): str | None
to_numpy(dtype: NpDtype | None, copy: bool, na_value): np.ndarray
to_parquet(path: FilePathOrBuffer | None, engine: str, compression: str | None, index: bool | None, partition_cols: list[str] | None, storage_options: StorageOptions): bytes | None
to_period(freq: Frequency | None, axis: Axis, copy: bool): DataFrame
to_records(index, column_dtypes, index_dtypes): np.recarray
to_stata(path: FilePathOrBuffer, convert_dates: dict[Hashable, str] | None, write_index: bool, byteorder: str | None, time_stamp: datetime.datetime | None, data_label: str | None, variable_labels: dict[Hashable, str] | None, version: int | None, convert_strl: Sequence[Hashable] | None, compression: CompressionOptions, storage_options: StorageOptions): None
to_string(buf: FilePathOrBuffer[str] | None, columns: Sequence[str] | None, col_space: int | None, header: bool | Sequence[str], index: bool, na_rep: str, formatters: fmt.FormattersType | None, float_format: fmt.FloatFormatType | None, sparsify: bool | None, index_names: bool, justify: str | None, max_rows: int | None, min_rows: int | None, max_cols: int | None, show_dimensions: bool, decimal: str, line_width: int | None, max_colwidth: int | None, encoding: str | None): str | None to_timestamp(freq: Frequency | None, how: str, axis: Axis, copy: bool): DataFrame
to_xml(path_or_buffer: FilePathOrBuffer | None, index: bool, root_name: str | None, row_name: str | None, na_rep: str | None, attr_cols: str | list[str] | None, elem_cols: str | list[str] | None, namespaces: dict[str | None, str] | None, prefix: str | None, encoding: str, xml_declaration: bool | None, pretty_print: bool | None, parser: str | None, stylesheet: FilePathOrBuffer | None, compression: CompressionOptions, storage_options: StorageOptions): str | None
transform(func: AggFuncType, axis: Axis): DataFrame
transpose(): DataFrame
unstack(level: Level, fill_value)
update(other, join: str, overwrite: bool, filter_func, errors: str): None
value_counts(subset: Sequence[Hashable] | None, normalize: bool, sort: bool, ascending: bool, dropna: bool)
where(cond, other, inplace, axis, level, errors, try_cast)

sparse plot

pandas.core.indexes.base.Index

__bool__
__hash__: NoneType
_accessors: set
_attributes: list[str]
_cache: dict
_can_hold_na: bool
_can_hold_strings: bool
_comparables: list[str]
_data: ExtensionArray | np.ndarray
_engine_type: type[libindex.IndexEngine]
_formatter_func
_has_complex_internals
_hidden_attrs: frozenset[str]
_id: object | None
_index_as_unique
_index_data
_is_numeric_dtype: bool
_is_strictly_monotonic_decreasing
_is_strictly_monotonic_increasing
_join_precedence: int
_na_value: float | NaTType
_name: Optional[Hashable]
_no_setting_name: bool
_requires_unique_msg: str
_supports_partial_string_indexing: bool
_typ: str
_values
asi8
get_level_values
has_duplicates
is_monotonic
is_monotonic_decreasing
is_monotonic_increasing
isnull

name

names: property
nlevels
notnull
shape
str
values

__abs__()
__and__(other)
__array__(dtype): np.ndarray
__array_wrap__(result, context)
__contains__(key: Any): bool
__copy__(): _IndexT
__deepcopy__(memo): _IndexT
__getitem__(key)
__iadd__(other)
__inv__()
__len__(): int
__neg__()
__new__(data, dtype, copy, name, tupleize_cols): Index
__nonzero__()
__or__(other)
__pos__()
__reduce__()
__repr__(): str_t
__setitem__(key, value)
__xor__(other)
_arith_method(other, op)
_assert_can_do_setop(other): bool
_can_hold_identifiers_and_holds_name(name): bool
_check_indexing_error(key)
_check_indexing_method(method: str_t | None, limit: int | None, tolerance): None
_cleanup(): None
_cmp_method(other, op)
_concat(to_concat: list[Index], name: Hashable): Index
_constructor(): type[_IndexT]
_convert_can_do_setop(other): tuple[Index, Hashable]
_convert_slice_indexer(key: slice, kind: str_t)
_convert_tolerance(tolerance, target: np.ndarray | Index): np.ndarray
_deprecated_arg(value, name: str_t, methodname: str_t): None
_difference(other, sort)
_dir_additions_for_owner(): set[str_t]
_drop_level_numbers(levnums: list[int])
_dtype_to_subclass(dtype: DtypeObj)
_engine(): libindex.IndexEngine
_ensure_array(data, dtype, copy: bool)
_filter_indexer_tolerance(target: Index | np.ndarray | ExtensionArray, indexer: np.ndarray, tolerance): np.ndarray
_find_common_type_compat(target): DtypeObj
_format_attrs(): list[tuple[str_t, str_t | int]]
_format_data(name): str_t
_format_duplicate_message(): DataFrame
_format_native_types(na_rep, quoting)
_format_space(): str_t
_format_with_header(header: list[str_t], na_rep: str_t): list[str_t]
_from_join_target(result: np.ndarray): ArrayLike
_get_attributes_dict(): dict[str_t, Any]
_get_engine_target(): np.ndarray
_get_fill_indexer(target: Index, method: str_t, limit: int | None, tolerance): np.ndarray
_get_fill_indexer_searchsorted(target: Index, method: str_t, limit: int | None): np.ndarray
_get_grouper_for_level(mapper, level)
_get_indexer(target: Index, method: str_t | None, limit: int | None, tolerance): np.ndarray
_get_indexer_non_comparable(target: Index, method, unique: Literal[True]): np.ndarray
_get_join_target(): np.ndarray
_get_level_number(level): int
_get_level_values(level): Index
_get_names(): FrozenList
_get_nearest_indexer(target: Index, limit: int | None, tolerance): np.ndarray
_get_partial_string_timestamp_match_key(key: T): T
_get_reconciled_name_object(other)
_get_string_slice(key: str_t)
_get_unique_index(): _IndexT
_get_values_for_loc(series: Series, loc, key)
_getitem_slice(slobj: slice): _IndexT
_inner_indexer(other: _IndexT): tuple[ArrayLike, np.ndarray, np.ndarray]
_intersection(other: Index, sort)
_intersection_via_get_indexer(other: Index, sort): ArrayLike
_invalid_indexer(form: str_t, key): TypeError
_is_all_dates(): bool
_is_comparable_dtype(dtype: DtypeObj): bool
_is_memory_usage_qualified(): bool
_is_multi(): bool
_isnan(): np.ndarray
_join_level(other: Index, level, how: str_t, keep_order: bool): tuple[MultiIndex, np.ndarray | None, np.ndarray | None]
_join_monotonic(other: Index, how: str_t)
_join_multi(other: Index, how: str_t)
_join_non_unique(other: Index, how: str_t): tuple[Index, np.ndarray, np.ndarray]
_left_indexer(other: _IndexT): tuple[ArrayLike, np.ndarray, np.ndarray]
_left_indexer_unique(other: _IndexT): np.ndarray
_maybe_cast_indexer(key)
_maybe_cast_listlike_indexer(target): Index
_maybe_cast_slice_bound(label, side: str_t, kind)
_maybe_check_unique(): None
_maybe_disable_logical_methods(opname: str_t)
_maybe_disallow_fill(allow_fill: bool, fill_value, indices): bool
_maybe_promote(other: Index): tuple[Index, Index]
_mpl_repr(): np.ndarray
_outer_indexer(other: _IndexT): tuple[ArrayLike, np.ndarray, np.ndarray]
_reindex_non_unique(target: Index): tuple[Index, np.ndarray, np.ndarray | None]
_rename(name: Hashable): _IndexT
_require_scalar(value)
_reset_identity(): None
_scalar_data_error(data)
_searchsorted_monotonic(label, side: str_t)
_set_names(values, level): None
_shallow_copy(values, name: Hashable): _IndexT
_should_compare(other: Index): bool
_should_fallback_to_positional(): bool
_simple_new(values, name: Optional[Hashable]): _IndexT
_sort_levels_monotonic(): _IndexT
_string_data_error(data)
_summary(name): str_t
_transform_index(func, level): Index
_unary_method(op)
_union(other: Index, sort)
_validate_can_reindex(indexer: np.ndarray): None
_validate_fill_value(value)
_validate_index_level(level): None
_validate_indexer(form: str_t, key, kind: str_t)
_validate_names(name, names, deep: bool): list[Hashable]
_validate_positional_slice(key: slice): None
_validate_sort_keyword(sort)
_view(): _IndexT
_wrap_intersection_result(other, result)
_wrap_joined_index(joined: ArrayLike, other: _IndexT): _IndexT
_wrap_setop_result(other: Index, result): Index
all()
any()
append(other: Index | Sequence[Index]): Index
argsort(): np.ndarray
array(): ExtensionArray
asof(label)
asof_locs(where: Index, mask: np.ndarray): np.ndarray
astype(dtype, copy)
copy(name: Hashable | None, deep: bool, dtype: Dtype | None, names: Sequence[Hashable] | None): _IndexT
delete(loc): _IndexT
difference(other, sort)
drop(labels, errors: str_t): Index
drop_duplicates(keep: str_t | bool): _IndexT
droplevel(level)
dropna(how: str_t): _IndexT
dtype(): DtypeObj
duplicated(keep: Literal['first', 'last', False]): np.ndarray
equals(other: Any): bool
fillna(value, downcast)
format(name: bool, formatter: Callable | None, na_rep: str_t): list[str_t]
get_indexer(target, method: str_t | None, limit: int | None, tolerance): np.ndarray
get_indexer_for(target): np.ndarray
get_indexer_non_unique(target): tuple[np.ndarray, np.ndarray]
get_loc(key, method, tolerance)
get_slice_bound(label, side: str_t, kind): int
get_value(series: Series, key)
groupby(values): PrettyDict[Hashable, np.ndarray]
hasnans(): bool
holds_integer(): bool
identical(other): bool
inferred_type(): str_t
insert(loc: int, item): Index
intersection(other, sort)
is_(other): bool
is_all_dates(): bool
is_boolean(): bool
is_categorical(): bool
is_floating(): bool
is_integer(): bool
is_interval(): bool
is_mixed(): bool
is_numeric(): bool
is_object(): bool
is_type_compatible(kind: str_t): bool
is_unique(): bool
isin(values, level): np.ndarray
isna(): np.ndarray
join(other, how: str_t, level, return_indexers: bool, sort: bool)
map(mapper, na_action)
memory_usage(deep: bool): int
notna(): np.ndarray
putmask(mask, value): Index
ravel(order)
reindex(target, method, level, limit, tolerance): tuple[Index, np.ndarray | None]
rename(name, inplace)
repeat(repeats, axis)
set_names(names, level, inplace: bool)
set_value(arr, key, value)
shift(periods, freq)
slice_indexer(start: Hashable | None, end: Hashable | None, step: int | None, kind: str_t | None): slice
slice_locs(start, end, step, kind)
sort()
sort_values(return_indexer: bool, ascending: bool, na_position: str_t, key: Callable | None)
sortlevel(level, ascending, sort_remaining)
symmetric_difference(other, result_name, sort)
take(indices, axis: int, allow_fill: bool, fill_value)
to_flat_index()
to_frame(index: bool, name: Hashable): DataFrame
to_native_types(slicer): np.ndarray
to_series(index, name: Hashable): Series
union(other, sort)
unique(level: Hashable | None): _IndexT
view(cls)
where(cond, other): Index

str

pandas.core.accessor.DirNamesMixin

_accessors: set[str]
_hidden_attrs: frozenset[str]

__dir__(): list[str]
_dir_additions(): set[str]
_dir_deletions(): set[str]

pandas.core.arraylike.OpsMixin

__add__(other)
__and__(other)
__divmod__(other)
__eq__(other)
__floordiv__(other)
__ge__(other)
__gt__(other)
__le__(other)
__lt__(other)
__mod__(other)
__mul__(other)
__ne__(other)
__or__(other)
__pow__(other)
__radd__(other)
__rand__(other)
__rdivmod__(other)
__rfloordiv__(other)
__rmod__(other)
__rmul__(other)
__ror__(other)
__rpow__(other)
__rsub__(other)
__rtruediv__(other)
__rxor__(other)
__sub__(other)
__truediv__(other)
__xor__(other)
_arith_method(other, op)
_cmp_method(other, op)
_logical_method(other, op)

pandas.core.base.IndexOpsMixin

T

__array_priority__: int
_hidden_attrs: frozenset[str]
_values
array

dtype
empty
is_monotonic
is_monotonic_decreasing
is_monotonic_increasing
is_unique
nbytes
ndim
shape
size
to_list

__iter__()
__len__(): int
_duplicated(keep: Literal['first', 'last', False]): np.ndarray
_map_values(mapper, na_action)
_memory_usage(deep: bool): int
_reduce(op, name: str)
argmax(axis, skipna: bool): int
argmin(axis, skipna): int
drop_duplicates(keep)
factorize(sort: bool, na_sentinel: int | None)
hasnans(): bool
isna()
item()
max(axis, skipna: bool)
min(axis, skipna: bool)
nunique(dropna: bool): int
searchsorted(value, side, sorter): np.ndarray
to_numpy(dtype: Dtype | None, copy: bool, na_value): np.ndarray
tolist()
transpose(): _T
unique()
value_counts(normalize: bool, sort: bool, ascending: bool, bins, dropna: bool)

pandas.core.base.PandasObject

_cache: dict[str,Any]
_constructor

__repr__(): str
__sizeof__(): int
_reset_cache(key: str | None): None

indexindex

pandas.core.generic.NDFrame

_AXIS_LEN: int
_AXIS_NAMES
_AXIS_NUMBERS
_AXIS_ORDERS: list[str]
_AXIS_REVERSED: bool
_AXIS_TO_AXIS_NUMBER: dict[Axis, int]
__array_priority__: int
__bool__
__hash__: NoneType
_accessors: set[str]
_attrs: dict[Hashable, Any]
_constructor
_data
_hidden_attrs: frozenset[str]
_info_axis
_info_axis_name: str
_info_axis_number: int
_internal_names: list[str]
_internal_names_set: set[str]
_is_copy: weakref.ReferenceType[NDFrame] | None
_is_mixed_type
_is_view
_item_cache: dict
_metadata: list[str]
_mgr

_stat_axis
_stat_axis_name: str
_stat_axis_number: int
_typ: str
_values
attrs
axes

backfill
dtypes
empty
flags
kurtosis
ndim
pad
product
shape
size
values

__abs__(): FrameOrSeries
__array__(dtype: NpDtype | None): np.ndarray
__array_ufunc__(ufunc: np.ufunc, method: str)
__array_wrap__(result: np.ndarray, context: tuple[Callable, tuple[Any,...], int] | None)
__contains__(key): bool_t
__copy__(deep: bool_t): FrameOrSeries
__deepcopy__(memo): FrameOrSeries
__delitem__(key): None
__finalize__(other, method: str | None): FrameOrSeries
__getattr__(name: str)
__getitem__(item)
__getstate__(): dict[str,Any]
__iadd__(other)
__iand__(other)
__ifloordiv__(other)
__imod__(other)
__imul__(other)
__init__(data: Manager, copy: bool_t, attrs: Mapping[Hashable, Any] | None)
__invert__()
__ior__(other)
__ipow__(other)
__isub__(other)
__iter__()
__itruediv__(other)
__ixor__(other)
__len__(): int
__neg__()
__nonzero__()
__pos__()
__repr__(): str
__round__(decimals: int): FrameOrSeries
__setattr__(name: str, value): None
__setstate__(state)
_accum_func(name: str, func, axis, skipna)
_add_numeric_operations()
_agg_by_level(name, axis, level, skipna)
_align_frame(other, join, axis, level, copy: bool_t, fill_value, method, limit, fill_axis)
_align_series(other, join, axis, level, copy: bool_t, fill_value, method, limit, fill_axis)
_as_manager(typ: str, copy: bool_t): FrameOrSeries
_check_inplace_and_allows_duplicate_labels(inplace)
_check_inplace_setting(value): bool_t
_check_is_chained_assignment_possible(): bool_t
_check_label_or_level_ambiguity(key, axis: int): None
_check_setitem_copy(stacklevel, t, force)
_clear_item_cache(): None
_clip_with_one_bound(threshold, method, axis, inplace)
_clip_with_scalar(lower, upper, inplace: bool_t)
_consolidate()
_consolidate_inplace(): None
_construct_axes_dict(axes)
_construct_axes_from_arguments(args, kwargs, require_all: bool_t, sentinel)
_convert(datetime: bool_t, numeric: bool_t, timedelta: bool_t): FrameOrSeries
_dir_additions(): set[str]
_drop_axis(labels, axis, level, errors: str): FrameOrSeries
_drop_labels_or_levels(keys, axis: int)
_find_valid_index(): Hashable | None
_from_mgr(mgr: Manager)
_get_axis(axis: Axis): Index
_get_axis_name(axis: Axis): str
_get_axis_number(axis: Axis): int
_get_axis_resolvers(axis: str): dict[str, Series | MultiIndex]
_get_block_manager_axis(axis: Axis): int
_get_bool_data()
_get_cleaned_column_resolvers(): dict[Hashable, Series]
_get_index_resolvers(): dict[Hashable, Series | MultiIndex]
_get_label_or_level_values(key: str, axis: int): np.ndarray
_get_numeric_data()
_indexed_same(other): bool_t
_init_mgr(mgr: Manager, axes, dtype: Dtype | None, copy: bool_t): Manager
_inplace_method(other, op)
_is_label_or_level_reference(key: str, axis: int): bool_t
_is_label_reference(key, axis): bool_t
_is_level_reference(key, axis)
_logical_func(name: str, func, axis, bool_only, skipna, level)
_maybe_update_cacher(clear: bool_t, verify_is_copy: bool_t): None
_min_count_stat_function(name: str, func, axis, skipna, level, numeric_only, min_count)
_needs_reindex_multi(axes, method, level): bool_t
_protect_consolidate(f)
_reindex_axes(axes, level, limit, tolerance, method, fill_value, copy): FrameOrSeries
_reindex_multi(axes, copy, fill_value)
_reindex_with_indexers(reindexers, fill_value, copy: bool_t, allow_dups: bool_t): FrameOrSeries
_repr_data_resource_()
_repr_latex_()
_reset_cacher(): None
_set_axis(axis: int, labels: Index): None
_set_axis_name(name, axis, inplace)
_set_axis_nocheck(labels, axis: Axis, inplace: bool_t)
_set_is_copy(ref: FrameOrSeries, copy: bool_t): None
_slice(slobj: slice, axis): FrameOrSeries
_stat_function(name: str, func, axis, skipna, level, numeric_only)
_stat_function_ddof(name: str, func, axis, skipna, level, ddof, numeric_only)
_take_with_is_copy(indices, axis): FrameOrSeries
_update_inplace(result, verify_is_copy: bool_t): None
_validate_dtype(dtype): DtypeObj | None
_where(cond, other, inplace, axis, level, errors)
abs(): FrameOrSeries
add_prefix(prefix: str): FrameOrSeries
add_suffix(suffix: str): FrameOrSeries
align(other, join, axis, level, copy, fill_value, method, limit, fill_axis, broadcast_axis)
all(axis, bool_only, skipna, level)
any(axis, bool_only, skipna, level)
asfreq(freq, method, how: str | None, normalize: bool_t, fill_value): FrameOrSeries
asof(where, subset)
astype(dtype, copy: bool_t, errors: str): FrameOrSeries
at_time(time, asof: bool_t, axis): FrameOrSeries
between_time(start_time, end_time, include_start: bool_t, include_end: bool_t, axis): FrameOrSeries
bfill(axis: None | Axis, inplace: bool_t, limit: None | int, downcast): FrameOrSeries | None
bool()
clip(lower, upper, axis: Axis | None, inplace: bool_t): FrameOrSeries | None
compare(other, align_axis: Axis, keep_shape: bool_t, keep_equal: bool_t)
convert_dtypes(infer_objects: bool_t, convert_string: bool_t, convert_integer: bool_t, convert_boolean: bool_t, convert_floating: bool_t): FrameOrSeries
copy(deep: bool_t): FrameOrSeries
cummax(axis, skipna)
cummin(axis, skipna)
cumprod(axis, skipna)
cumsum(axis, skipna)
describe(percentiles, include, exclude, datetime_is_numeric): FrameOrSeries
drop(labels, axis, index, columns, level, inplace: bool_t, errors: str)
droplevel(level, axis): FrameOrSeries
equals(other: object): bool_t
ewm(com: float | None, span: float | None, halflife: float | TimedeltaConvertibleTypes | None, alpha: float | None, min_periods: int | None, adjust: bool_t, ignore_na: bool_t, axis: Axis, times: str | np.ndarray | FrameOrSeries | None): ExponentialMovingWindow expanding(min_periods: int, center: bool_t | None, axis: Axis, method: str): Expanding
ffill(axis: None | Axis, inplace: bool_t, limit: None | int, downcast): FrameOrSeries | None
fillna(value, method, axis, inplace: bool_t, limit, downcast): FrameOrSeries | None
filter(items, like: str | None, regex: str | None, axis): FrameOrSeries
first(offset): FrameOrSeries
first_valid_index(): Hashable | None
get(key, default)
head(n: int): FrameOrSeries
infer_objects(): FrameOrSeries
interpolate(method: str, axis: Axis, limit: int | None, inplace: bool_t, limit_direction: str | None, limit_area: str | None, downcast: str | None): FrameOrSeries | None
isna(): FrameOrSeries
isnull(): FrameOrSeries
items()
iteritems()
keys()
kurt(axis, skipna, level, numeric_only)
last(offset): FrameOrSeries
last_valid_index(): Hashable | None
mad(axis, skipna, level)
mask(cond, other, inplace, axis, level, errors, try_cast)
max(axis, skipna, level, numeric_only)
mean(axis, skipna, level, numeric_only)
median(axis, skipna, level, numeric_only)
min(axis, skipna, level, numeric_only)
notna(): FrameOrSeries
notnull(): FrameOrSeries
pct_change(periods, fill_method, limit, freq): FrameOrSeries
pipe(func: Callable[..., T] | tuple[Callable[..., T], str]): T
pop(item: Hashable): Series | Any
prod(axis, skipna, level, numeric_only, min_count)
rank(axis, method: str, numeric_only: bool_t | None, na_option: str, ascending: bool_t, pct: bool_t): FrameOrSeries
reindex(): FrameOrSeries
reindex_like(other, method: str | None, copy: bool_t, limit, tolerance): FrameOrSeries
rename(mapper: Renamer | None): FrameOrSeries | None
rename_axis(mapper)
replace(to_replace, value, inplace: bool_t, limit: int | None, regex, method)
resample(rule, axis, closed: str | None, label: str | None, convention: str, kind: str | None, loffset, base: int | None, on, level, origin: str | TimestampConvertibleTypes, offset: TimedeltaConvertibleTypes | None): Resampler rolling(window: int | timedelta | BaseOffset | BaseIndexer, min_periods: int | None, center: bool_t, win_type: str | None, on: str | None, axis: Axis, closed: str | None, method: str)
sample(n, frac: float | None, replace: bool_t, weights, random_state, axis: Axis | None, ignore_index: bool_t): FrameOrSeries
sem(axis, skipna, level, ddof, numeric_only)
set_axis(labels, axis: Axis, inplace: Literal[False]): FrameOrSeries
set_flags(): FrameOrSeries
shift(periods, freq, axis, fill_value): FrameOrSeries
skew(axis, skipna, level, numeric_only)
slice_shift(periods: int, axis): FrameOrSeries
sort_index(axis, level, ascending: bool_t | int | Sequence[bool_t | int], inplace: bool_t, kind: str, na_position: str, sort_remaining: bool_t, ignore_index: bool_t, key: IndexKeyFunc)
sort_values(axis, ascending, inplace: bool_t, kind: str, na_position: str, ignore_index: bool_t, key: ValueKeyFunc)
squeeze(axis)
std(axis, skipna, level, ddof, numeric_only)
sum(axis, skipna, level, numeric_only, min_count)
swapaxes(axis1, axis2, copy): FrameOrSeries
tail(n: int): FrameOrSeries
take(indices, axis, is_copy: bool_t | None): FrameOrSeries
to_clipboard(excel: bool_t, sep: str | None): None
to_csv(path_or_buf: FilePathOrBuffer[AnyStr] | None, sep: str, na_rep: str, float_format: str | None, columns: Sequence[Hashable] | None, header: bool_t | list[str], index: bool_t, index_label: IndexLabel | None, mode: str, encoding: str | None, compression: CompressionOptions, quoting: int | None, quotechar: str, line_terminator: str | None, chunksize: int | None, date_format: str | None, doublequote: bool_t, escapechar: str | None, decimal: str, errors: str, storage_options: StorageOptions): str | None to_excel(excel_writer, sheet_name: str, na_rep: str, float_format: str | None, columns, header, index, index_label, startrow, startcol, engine, merge_cells, encoding, inf_rep, verbose, freeze_panes, storage_options: StorageOptions): None
to_hdf(path_or_buf, key: str, mode: str, complevel: int | None, complib: str | None, append: bool_t, format: str | None, index: bool_t, min_itemsize: int | dict[str, int] | None, nan_rep, dropna: bool_t | None, data_columns: bool_t | list[str] | None, errors: str, encoding: str): None
to_json(path_or_buf: FilePathOrBuffer | None, orient: str | None, date_format: str | None, double_precision: int, force_ascii: bool_t, date_unit: str, default_handler: Callable[[Any], JSONSerializable] | None, lines: bool_t, compression: CompressionOptions, index: bool_t, indent: int | None, storage_options: StorageOptions): str | None
to_latex(buf, columns, col_space, header, index, na_rep, formatters, float_format, sparsify, index_names, bold_rows, column_format, longtable, escape, encoding, decimal, multicolumn, multicolumn_format, multirow, caption, label, position)
to_pickle(path, compression: CompressionOptions, protocol: int, storage_options: StorageOptions): None
to_sql(name: str, con, schema, if_exists: str, index: bool_t, index_label, chunksize, dtype: DtypeArg | None, method): None
to_xarray()
truncate(before, after, axis, copy: bool_t): FrameOrSeries
tshift(periods: int, freq, axis: Axis): FrameOrSeries
tz_convert(tz, axis, level, copy: bool_t): FrameOrSeries
tz_localize(tz, axis, level, copy: bool_t, ambiguous, nonexistent: str): FrameOrSeries
var(axis, skipna, level, ddof, numeric_only)
where(cond, other, inplace, axis, level, errors, try_cast)
xs(key, axis, level, drop_level: bool_t)

submenus.single_corpus.SingleCorpusMenu

__anCfg: dict
__dataDic: dict[str:pd.DataFrame()]
__debug: bool
__prefix: str
__rephrase_df: NoneType
__rephrase_old
criteria: str

__checkSessionState(): bool
__corporaPickerChckBox()
__init__(dataDic: dict[str:pd.DataFrame()], prefix: str, anType: str): None
__update_block(name: str)
__update_corpora_checkbox()
cleanSelections()
container(s: str, units_choice: str)
getCriteria(): str
getDF(): pd.DataFrame()
sidebar()
tab(units)

__rephrase_df __rephrase_old __rephrase_df __rephrase_df

pandas.core.indexing.IndexingMixin

at
iat
iloc
loc

index columns columnsindex

pandas.core.indexes.frozen.FrozenList

__add__
__delitem__
__delslice__
__iadd__
__imul__
__req__

__setitem__
__setslice__
append
extend
insert
pop
remove
sort

__eq__(other: Any): bool
__getitem__(n)
__hash__()
__mul__(other)
__radd__(other)
__reduce__()
__repr__(): str
__str__(): str
_disabled()
difference(other): FrozenList
union(other): FrozenList

pandas.core.indexes.multi.MultiIndex

__abs__
__add__
__divmod__
__floordiv__
__iadd__
__inv__
__isub__
__mod__
__mul__
__neg__
__pos__
__pow__

__radd__
__rdivmod__
__rfloordiv__
__rmod__
__rmul__
__rpow__

__rsub__
__rtruediv__
__sub__
__truediv__
_cache: dict
_codes
_comparables: list
_constructor
_duplicated
_has_complex_internals
_hidden_attrs
_is_all_dates
_levels
_names
_typ: str
array
codes
levshape
lexsort_depth
names: property
nlevels

rename

sortorder: int | None
values

__array__(dtype): np.ndarray
__contains__(key: Any): bool
__getitem__(key)
__len__(): int
__new__(levels, codes, sortorder, names, dtype, copy, name, verify_integrity: bool)
__reduce__()
_check_indexing_error(key)
_convert_can_do_setop(other)
_convert_listlike_indexer(keyarr): np.ndarray | None
_difference(other, sort): MultiIndex
_drop_from_level(codes, level, errors): MultiIndex
_engine()
_format_native_types(na_rep)
_formatter_func(tup)
_get_codes_for_sorting(): list[Categorical]
_get_grouper_for_level(mapper, level)
_get_indexer(target: Index, method: str | None, limit: int | None, tolerance): np.ndarray
_get_level_indexer(key, level: int, indexer)
_get_level_number(level): int
_get_level_values(level: int, unique: bool): Index
_get_loc_level(key, level: int | list[int], drop_level: bool)
_get_loc_single_level_index(level_index: Index, key: Hashable): int
_get_names(): FrozenList
_get_partial_string_timestamp_match_key(key)
_get_reconciled_name_object(other): MultiIndex
_get_values_for_loc(series: Series, loc, key)
_getitem_slice(slobj: slice): MultiIndex
_inferred_type_levels(): list[str]
_is_comparable_dtype(dtype: DtypeObj): bool
_is_lexsorted(): bool
_is_memory_usage_qualified(): bool
_lexsort_depth(): int
_maybe_match_names(other)
_nbytes(deep: bool): int
_partial_tup_index(tup: tuple, side)
_reorder_indexer(seq: tuple[Scalar | Iterable | AnyArrayLike,...], indexer: Int64Index): Int64Index
_set_codes(codes, level, copy: bool, validate: bool, verify_integrity: bool): None
_set_levels(levels, level, copy: bool, validate: bool, verify_integrity: bool): None
_set_names(names, level, validate: bool)
_shallow_copy(values: np.ndarray, name): MultiIndex
_should_fallback_to_positional(): bool
_sort_levels_monotonic(): MultiIndex
_union(other, sort): MultiIndex
_validate_codes(level: list, code: list)
_validate_fill_value(item)
_values(): np.ndarray
_verify_integrity(codes: list | None, levels: list | None)
_view(): MultiIndex
_wrap_intersection_result(other, result)
append(other)
argsort(): np.ndarray
astype(dtype, copy: bool)
copy(names, dtype, levels, codes, deep, name)
delete(loc): MultiIndex
drop(codes, level, errors)
drop_duplicates(keep: str | bool): MultiIndex
dropna(how: str): MultiIndex
dtype(): np.dtype
dtypes(): Series
duplicated(keep): np.ndarray
equal_levels(other: MultiIndex): bool
equals(other: object): bool
fillna(value, downcast)
format(name: bool | None, formatter: Callable | None, na_rep: str | None, names: bool, space: int, sparsify, adjoin: bool): list
from_arrays(arrays, sortorder, names): MultiIndex
from_frame(df: DataFrame, sortorder, names): MultiIndex
from_product(iterables, sortorder, names): MultiIndex
from_tuples(tuples: Iterable[tuple[Hashable,...]], sortorder: int | None, names: Sequence[Hashable] | None): MultiIndex
get_level_values(level)
get_loc(key, method)
get_loc_level(key, level, drop_level: bool)
get_locs(seq)
get_slice_bound(label: Hashable | Sequence[Hashable], side: str, kind: str | None): int
inferred_type(): str
insert(loc: int, item): MultiIndex
is_lexsorted(): bool
is_monotonic_decreasing(): bool
is_monotonic_increasing(): bool
isin(values, level): np.ndarray
levels(): FrozenList
memory_usage(deep: bool): int
nbytes(): int
reindex(target, method, level, limit, tolerance): tuple[MultiIndex, np.ndarray | None]
remove_unused_levels(): MultiIndex
reorder_levels(order): MultiIndex
repeat(repeats: int, axis): MultiIndex
set_codes(codes, level, inplace, verify_integrity: bool)
set_levels(levels, level, inplace, verify_integrity: bool)
set_names(names, level, inplace: bool): MultiIndex | None
slice_locs(start, end, step, kind)
sortlevel(level, ascending: bool, sort_remaining: bool): tuple[MultiIndex, np.ndarray]
swaplevel(i, j): MultiIndex
take(indices, axis: int, allow_fill: bool, fill_value): MultiIndex
to_flat_index(): Index
to_frame(index: bool, name): DataFrame
truncate(before, after): MultiIndex
unique(level)
view(cls)

_codes _levels

index

pandas.core.indexes.range.RangeIndex

_cache: dict
_deprecation_message: str
_dtype_validation_metadata: tuple
_engine_type
_int64index
_name: Optional[Hashable]
_range: range
_start
_step
_stop
_typ: str
dtype
inferred_type
is_unique
size
start

step
stop

__contains__(key: Any): bool
__floordiv__(other)
__getitem__(key)
__iter__()
__len__(): int
__new__(start, stop, step, dtype: Dtype | None, copy: bool, name: Hashable): RangeIndex
__reduce__()
_arith_method(other, op)
_cached_int64index(): Int64Index
_cmp_method(other, op)
_concat(indexes: list[Index], name: Hashable): Index
_constructor(): type[Int64Index]
_data(): np.ndarray
_difference(other, sort)
_extended_gcd(a: int, b: int): tuple[int, int, int]
_format_attrs()
_format_data(name)
_format_with_header(header: list[str], na_rep: str): list[str]
_get_data_as_items()
_get_indexer(target: Index, method: str | None, limit: int | None, tolerance): np.ndarray
_getitem_slice(slobj: slice): RangeIndex
_intersection(other: Index, sort)
_min_fitting_element(lower_limit: int): int
_minmax(meth: str)
_shallow_copy(values, name: Hashable)
_simple_new(values: range, name: Optional[Hashable]): RangeIndex
_union(other: Index, sort)
_view(): RangeIndex
all(): bool
any(): bool
argsort(): np.ndarray
copy(name: Hashable, deep: bool, dtype: Dtype | None, names)
delete(loc): Int64Index
equals(other: object): bool
factorize(sort: bool, na_sentinel: int | None): tuple[np.ndarray, RangeIndex]
from_range(data: range, name, dtype: Dtype | None): RangeIndex
get_loc(key, method, tolerance)
is_monotonic_decreasing(): bool
is_monotonic_increasing(): bool
max(axis, skipna: bool): int
memory_usage(deep: bool): int
min(axis, skipna: bool): int
nbytes(): int
repeat(repeats, axis): Int64Index
symmetric_difference(other, result_name: Hashable, sort)
take(indices, axis: int, allow_fill: bool, fill_value): Int64Index
tolist(): list[int]

index

__single_corpora_menu __cmp_corpora_menu